Qwythos-gguf

Hugging Face | Search models, datasets, users... | Models | Datasets | Spaces | Buckets NEW | Docs | Enterprise | Pricing

Buckets: Matuxy / Qwythos-9B-Claude-Mythos-5-1M-GGUF-bucket private

Files xet | Settings

0 Bytes | 15 files | Updated less than a minute ago | Go to file Ctrl+K | Upload file(s)

Name	Size	Uploaded	Xet hash
.gitattributes	2.47 kB xet	less than a minute ago	3f51bd66
Qwythos-9B-Claude-Mythos-5-1M-BF16.gguf	17.9 GB xet	less than a minute ago	4552ce52
Qwythos-9B-Claude-Mythos-5-1M-MTP-BF16.gguf	18.4 GB xet	less than a minute ago	a68d8b08
Qwythos-9B-Claude-Mythos-5-1M-MTP-Q4_K_M.gguf	5.89 GB xet	less than a minute ago	89294873
Qwythos-9B-Claude-Mythos-5-1M-MTP-Q5_K_M.gguf	6.73 GB xet	less than a minute ago	18f8ba75
Qwythos-9B-Claude-Mythos-5-1M-MTP-Q6_K.gguf	7.62 GB xet	less than a minute ago	559f752b
Qwythos-9B-Claude-Mythos-5-1M-MTP-Q8_0.gguf	9.79 GB xet	less than a minute ago	e0865ace
Qwythos-9B-Claude-Mythos-5-1M-Q4_K_M.gguf	5.63 GB xet	less than a minute ago	327ba44d
Qwythos-9B-Claude-Mythos-5-1M-Q5_K_M.gguf	6.47 GB xet	less than a minute ago	52e607cd
Qwythos-9B-Claude-Mythos-5-1M-Q6_K.gguf	7.36 GB xet	less than a minute ago	d447a5e1
Qwythos-9B-Claude-Mythos-5-1M-Q8_0.gguf	9.53 GB xet	less than a minute ago	3dffcf96
README.md	10.3 kB xet	less than a minute ago	2982ae50
SHA256SUMS	1.31 kB xet	less than a minute ago	31ae6d93
TEST_REPORT.md	3.72 kB xet	less than a minute ago	2de3f342
mmproj-Qwythos-9B-Claude-Mythos-5-1M-F16.gguf	918 MB xet	less than a minute ago	58f3899f

Sync this bucket | Mount this bucket

Total size 0 Bytes
Files 15
Last updated Jul 25
Pre-warmed CDN US EU US EU

Contributors

README.md

Qwythos-9B-Claude-Mythos-5-1M-GGUF

Qwythos-9B-Claude-Mythos-5-1M-GGUF

Developed by Matuxyz

GGUF quantizations of **empero-ai/Qwythos-9B-Claude-Mythos-5-1M** for **llama.cpp**, Ollama, LM Studio, jan, KoboldCpp, and other GGUF runtimes.

Qwythos-9B is a full-parameter reasoning model post-trained on over 500 million tokens of high-quality Claude Mythos / Claude Fable traces with chain-of-thought generated in-house by Empero AI's internal `rethink` tool. It dominates the base Qwen3.5-9B under matched evaluation (**+34 pts MMLU, +30 pts gsm8k-strict, +19 pts gsm8k-flex**), supports **native function calling** per the Qwen3.5 spec, and ships with a **1,048,576-token (1M) context window** via YaRN rope-scaling enabled by default.

For full training details, evaluation numbers, and capability writeup, see the **base model card**.

Files

Normal text weights — fixed v3 replacements

File	Quant	Size	Notes
`Qwythos-9B-Claude-Mythos-5-1M-Q4_K_M.gguf`	Q4_K_M	5.24 GiB / 5.63 GB	**recommended default** — fixed v3, best compatibility
`Qwythos-9B-Claude-Mythos-5-1M-Q5_K_M.gguf`	Q5_K_M	6.02 GiB / 6.47 GB	fixed v3, balanced quality / size
`Qwythos-9B-Claude-Mythos-5-1M-Q6_K.gguf`	Q6_K	6.85 GiB / 7.36 GB	fixed v3, high quality
`Qwythos-9B-Claude-Mythos-5-1M-Q8_0.gguf`	Q8_0	8.87 GiB / 9.53 GB	fixed v3, near-lossless
`Qwythos-9B-Claude-Mythos-5-1M-BF16.gguf`	BF16	16.69 GiB / 17.92 GB	fixed v3, full precision conversion base

If you don't know which to pick, **Q4_K_M is the right starting point** — it's the smallest practical quant with good quality preservation.

MTP-enabled text weights — fixed v3 variants

These include the restored Qwen3.5-compatible MTP head inside the GGUF. Use them with llama.cpp builds that support MTP draft speculation, for example `--spec-type draft-mtp`.

File	Quant	Size	Notes
`Qwythos-9B-Claude-Mythos-5-1M-MTP-Q4_K_M.gguf`	Q4_K_M + MTP	5.48 GiB / 5.89 GB	**recommended MTP default**
`Qwythos-9B-Claude-Mythos-5-1M-MTP-Q5_K_M.gguf`	Q5_K_M + MTP	6.26 GiB / 6.73 GB	MTP, balanced quality / size
`Qwythos-9B-Claude-Mythos-5-1M-MTP-Q6_K.gguf`	Q6_K + MTP	7.09 GiB / 7.62 GB	MTP, high quality
`Qwythos-9B-Claude-Mythos-5-1M-MTP-Q8_0.gguf`	Q8_0 + MTP	9.11 GiB / 9.79 GB	MTP, near-lossless
`Qwythos-9B-Claude-Mythos-5-1M-MTP-BF16.gguf`	BF16 + MTP	17.14 GiB / 18.41 GB	MTP, full precision conversion base

Vision projector — for image input

File	Size	Notes
`mmproj-Qwythos-9B-Claude-Mythos-5-1M-F16.gguf`	0.86 GiB / 0.92 GB	CLIP-style vision encoder + projector; **required for images**, pairs with any normal or MTP quant above

Qwythos inherits its **vision tower from the Qwen3.5-9B base model** — the vision path was *frozen* during SFT (training was text-only), so the vision behavior is identical to base Qwen3.5-9B's multimodal capability. The mmproj is interchangeable with any community-built Qwen3.5-9B `mmproj-*.gguf`.

Quick start

llama.cpp (`llama-cli`)

```
llama-cli \
  -m Qwythos-9B-Claude-Mythos-5-1M-Q4_K_M.gguf \
  -p "Walk through the biochemistry of how organophosphate ne
rve agents inhibit acetylcholinesterase." \
  -n 8192 \
  --temp 0.6 --top-p 0.95 --top-k 20 --repeat-penalty 1.05 \
  -c 16384
```

Ollama

```
ollama run hf.co/empero-ai/Qwythos-9B-Claude-Mythos-5-1M-GGUF:Q4_K_M
```

LM Studio / jan / KoboldCpp

Drop any of the `.gguf` files into your runtime's model directory. Qwythos uses the standard Qwen3.5 chat template; modern GGUF runtimes load it automatically from the file.

llama.cpp with MTP draft speculation

```
llama-server \
  -m Qwythos-9B-Claude-Mythos-5-1M-MTP-Q4_K_M.gguf \
  --spec-type draft-mtp \
  --spec-draft-n-max 6 \
  -c 16384 --port 8080
```

MTP support requires a recent llama.cpp build. If your runtime does not support MTP yet, use the normal fixed v3 files above.

Vision (image input)

Qwythos supports **image input** out of the box. Download both a text quant and the `mmproj-*.gguf` file from this repo, then run with llama.cpp's multimodal CLI or server.

llama.cpp (`llama-mtmd-cli`)

```
llama-mtmd-cli \
  -m Qwythos-9B-Claude-Mythos-5-1M-Q4_K_M.gguf \
  --mmproj mmproj-Qwythos-9B-Claude-Mythos-5-1M-F16.gguf \
  --image ./photo.jpg \
  -p "Describe this image in detail." \
```

```
  --temp 0.6 --top-p 0.95 --top-k 20 \
  -c 16384
```

llama.cpp server (OpenAI-compatible API with images)

```
llama-server \
  -m Qwythos-9B-Claude-Mythos-5-1M-Q4_K_M.gguf \
  --mmproj mmproj-Qwythos-9B-Claude-Mythos-5-1M-F16.gguf \
  -c 16384 --port 8080
```

Then POST to `/v1/chat/completions` with an image URL or base64 payload — the standard OpenAI vision API shape works.

LM Studio

Load the text quant; LM Studio detects the matching `mmproj-*.gguf` in the same folder and enables the image-attach button automatically.

What vision unlocks

Since Qwythos inherits its vision tower unchanged from Qwen3.5-9B base, expect Qwen3.5-9B's documented vision capabilities: detailed image description, OCR (printed + handwritten), chart/table reading, UI/document understanding, basic spatial reasoning.

Honest note: the SFT used to produce Qwythos was **text-only** — we did not fine-tune the vision tower or train on any image-paired data. Image-grounded reasoning therefore inherits the base model's behavior; it has not been independently evaluated as part of this release. If your application is *primarily* vision-driven, validate on your own use case first.

Sampling recommendations

Qwythos is a reasoning model — every response opens with a `<think>...</think>` block before the final answer. Use these settings as defaults:

Parameter	Value
`temperature`	0.6
`top_p`	0.95
`top_k`	20
`repeat_penalty`	1.05
`max_new_tokens`	16384 (generous budget for `<think>` + answer)

These match Qwen3.5's official thinking-mode recommendations. **Avoid greedy decoding and very-low-temperature sampling (T ≤ 0.3)** — both can cause repetition loops on long reasoning generations.

Long context (1M tokens)

The GGUFs ship with YaRN rope-scaling baked in for a **1,048,576-token context window** (4× extension over the 262k native).

To use the full 1M window in `llama-cli`, set `-c 1010000` (or any context length up to that). For shorter prompts, lower `-c` to reduce KV-cache memory — at default settings llama.cpp will autosize.

A single H100/H200-class GPU comfortably handles **256k–512k**; the full 1M typically needs tensor-parallel multi-GPU or aggressive KV-cache offload.

Capabilities (from the base model card)

- **+34 pts MMLU, +30 pts gsm8k-strict, +19 pts gsm8k-flex** vs. base Qwen3.5-9B under matched lm-eval-harness evaluation
- **Native function calling** per Qwen3.5's chat-template spec — emits `<tool_call><function=NAME><parameter=NAME>VAL</parameter></function></tool_call>` blocks ready for any tool-use loop
- **Self-correcting with tools**: in a 7-prompt tool-use harness (Python executor + DuckDuckGo search), Qwythos produced source-cited correct answers on 7/7, including 4/4 closed-book failure-modes from the original review
- **Uncensored** — engages seriously with technically demanding questions across cybersecurity, red-teaming, biology, pharmacology, and clinical

medicine

- **1,048,576-token (1M) context** — YaRN rope-scaling enabled by default

For full eval transcripts and per-task numbers, see the **base model card's `evals/` folder**.

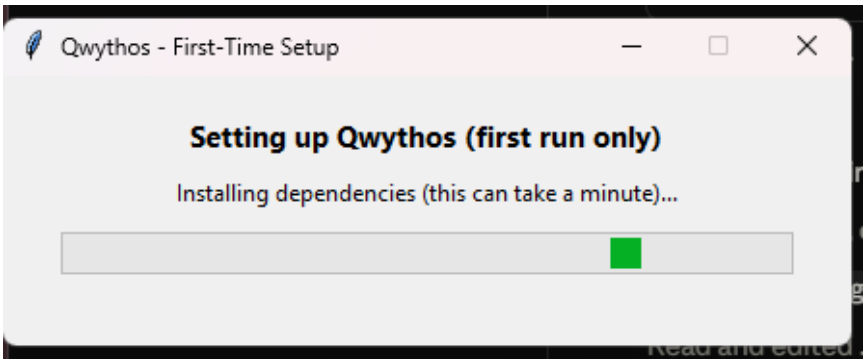
Limitations

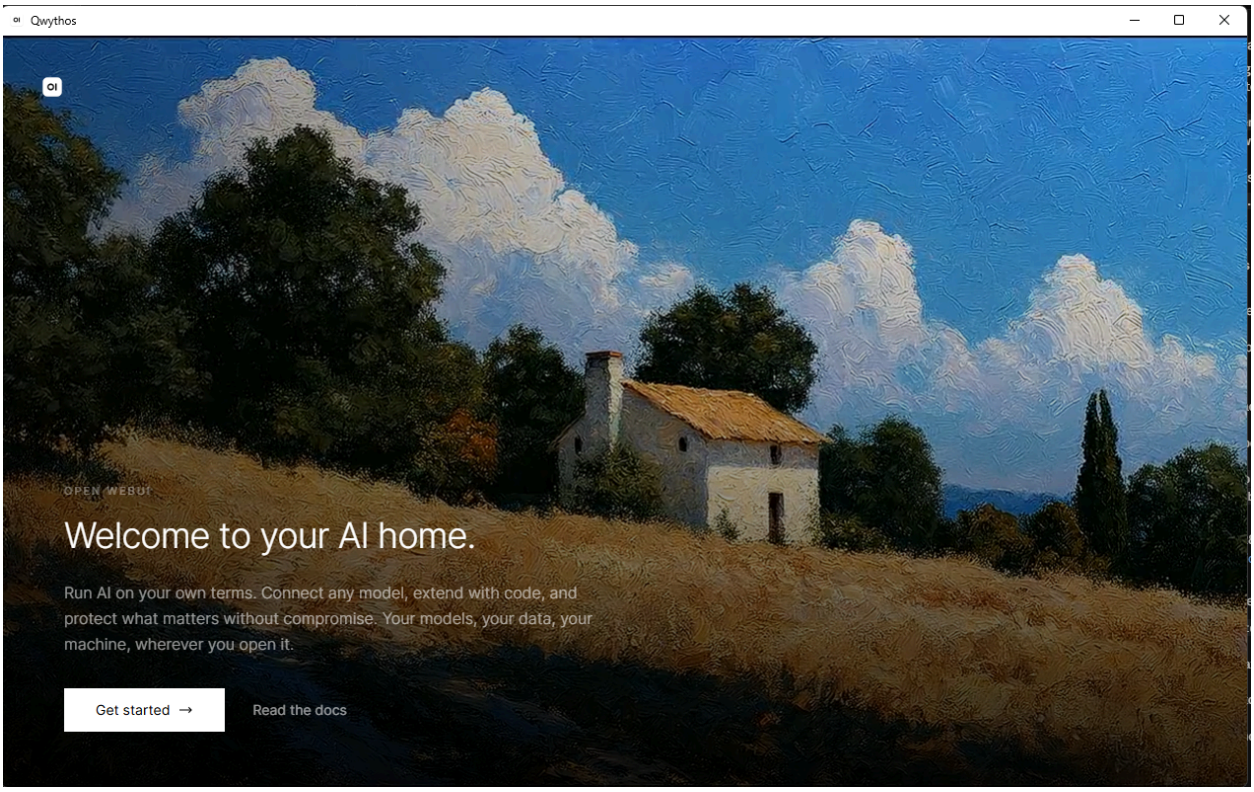
- **Reasoning model.** Every answer opens with a `<think>` block; allow generous `max_new_tokens` and parse/strip `<think>...</think>` for end users.
- **Use recommended sampling.** Greedy / very-low-temp can cause repetition loops.
- **Verify specifics in safety-critical contexts.** Like all closed-book LLMs in this weight class, Qwythos can over-commit to specific identifiers (CVEs, hashcat modes, drug positions) it isn't certain about. Pair with retrieval or function calling in such deployments — the model uses tools cleanly when offered them.
- **Uncensored — add your own application-level review/safety layer** for end-user-facing deployments where that matters.

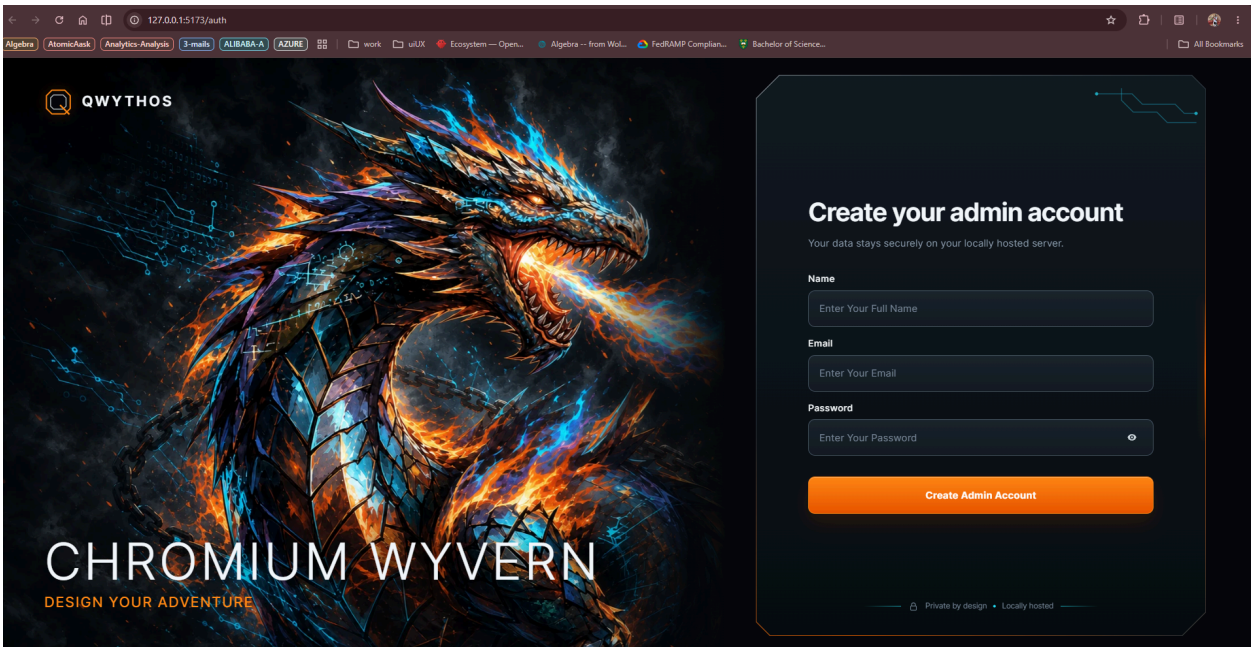
Acknowledgements

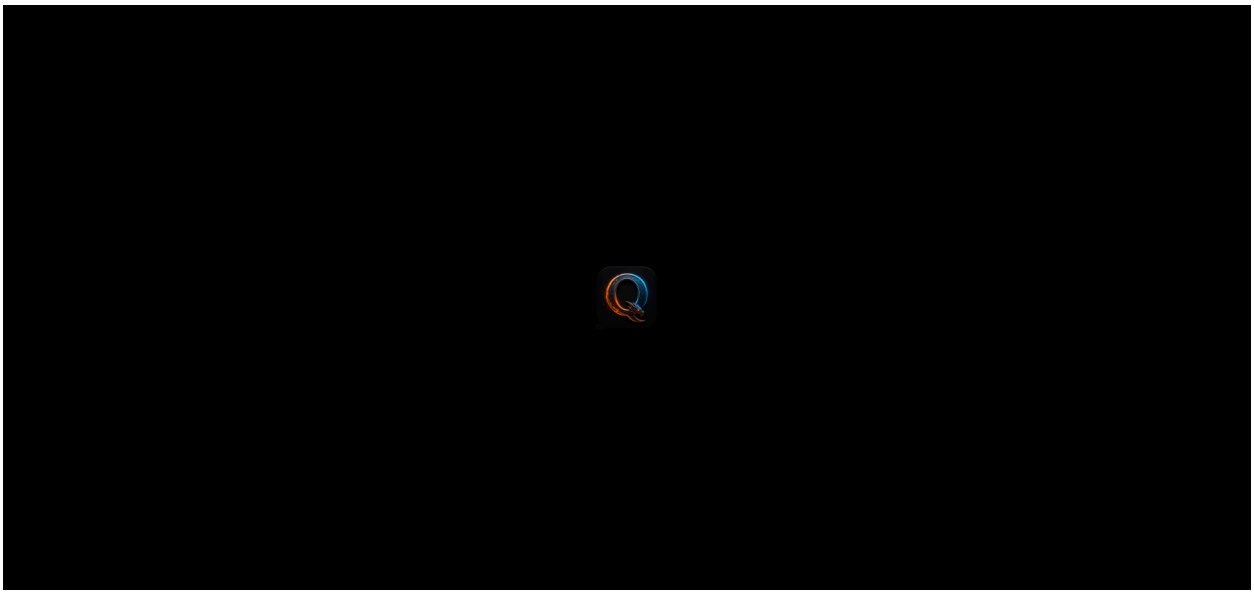
- Developed and released by Matuxyz in collab
- Base model: **Qwen3.5-9B** (Alibaba Qwen team)
- Quantization: **llama.cpp** (ggml-org)
- Vision projector (`mmproj`): inherited from Qwen3.5-9B (vision tower unchanged); F16 GGUF re-hosted with thanks to **Unsloth** for the original conversion

Matuxy/Qwythos-9B-Claude-Mythos-5-1M-GGUF-bucket

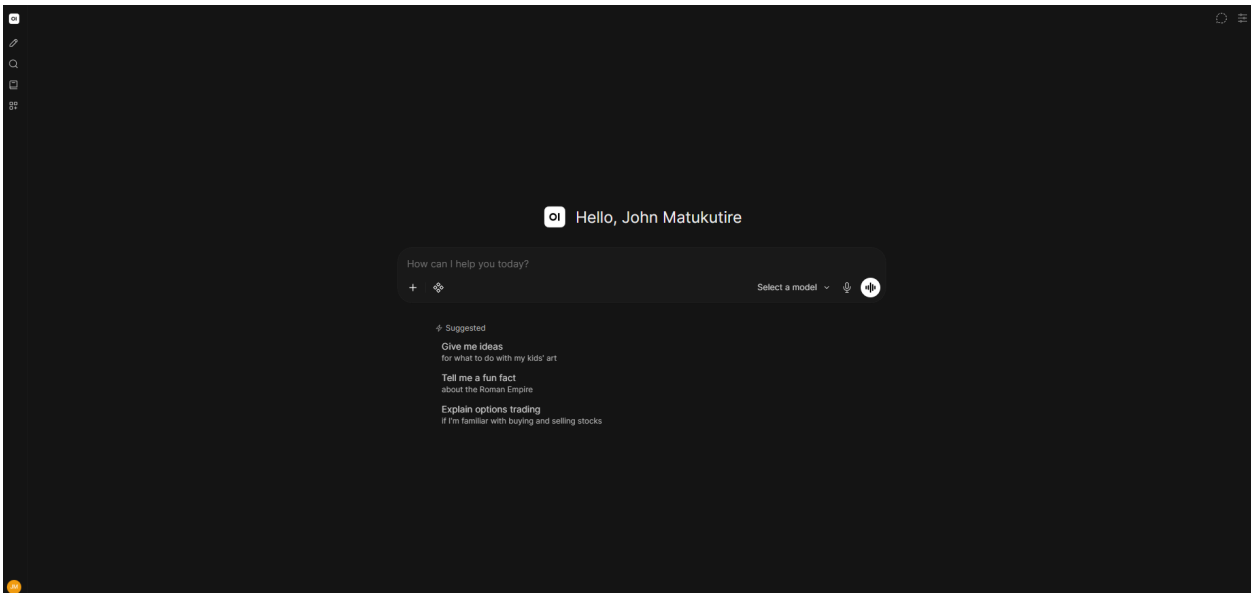

Qwythos - First-Time Setup
Setting up Qwythos (first run only)
Installing dependencies (this can take a minute)...


Qwythos
OPEN WEBUI
Welcome to your AI home.
Run AI on your own terms. Connect any model, extend with code, and protect what matters without compromise. Your models, your data, your machine, wherever you open it.
Get started →
Read the docs

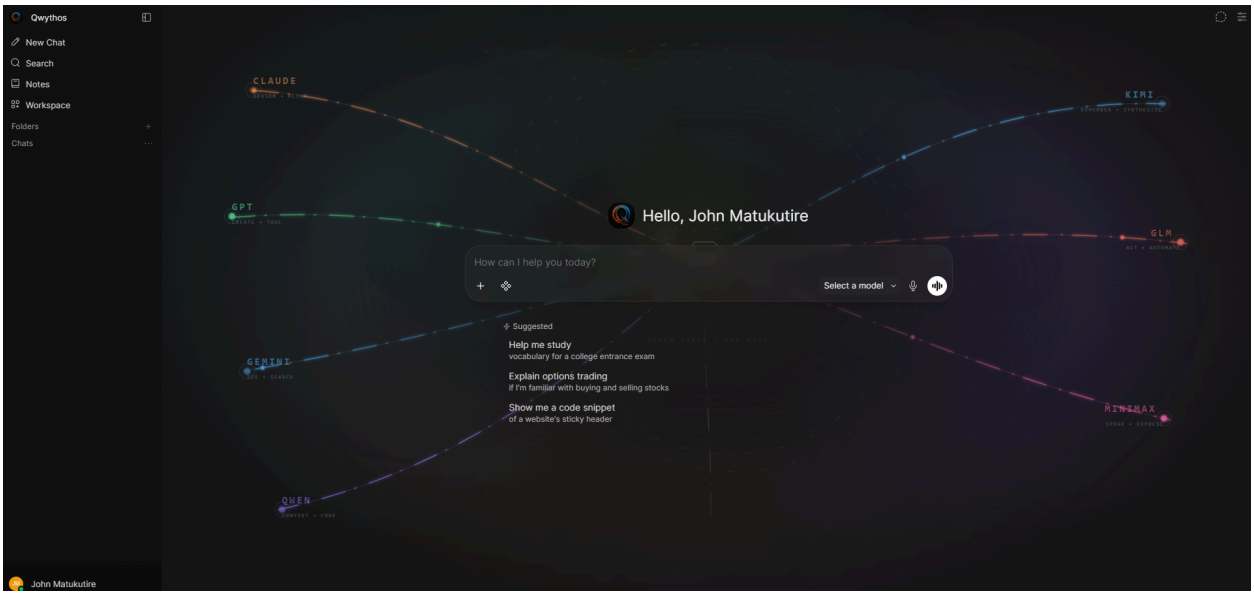




can we design a qythos background animation inspired by the frontier llms in my stack you included


Hello, John Matukutire
How can I help you today?
Select a model
Suggested
Give me ideas
for what to do with my kids' art
Tell me a fun fact
about the Roman Empire
Explain options trading
if I'm familiar with buying and selling stocks


QWYTHOS
REASON • IMAGINE • CONVERGE
理
REASON
想
IMAGINE
合
CONVERGE
源
SOURCE
CLAUDE
REASONING • ANALYSIS
SAFETY • ALIGNMENT
GPT
VERSATILITY • TOOLS
CREATIVITY • CODING
GEMINI
MULTIMODALITY • VISION
RESEARCH • REALTIME
QWEN
EFFICIENCY • LONG CONTEXT
MULTILINGUAL • REASONING
KIMI
LONG CONTEXT • MEMORY
SUMMARIZATION • KNOWLEDGE
GLM
FUNCTION CALLING
AGENTIC • AUTOMATION
MINIMAX
SPEECH • AUDIO
REALTIME • EXPRESSIVE
THE STACK
OPEN WEIGHTS • OPEN FUTURE
BUILT FOR THE HIVE
SEVEN MINDS • ONE HIVE • INFINITE CONTEXT



Qwythos Architecture

> **REASON · IMAGINE · CONVERGE** *Seven Minds · One Hive · Infinite Context*

This document maps the full architecture of the Qwythos platform — a self-hosted, multi-model AI orchestration system that unifies multiple LLM providers behind a single extensible interface.

Table of Contents

- Security Model
- Storage & Data Layer
- Infrastructure & Deployment
- Integration Points
- Directory Map

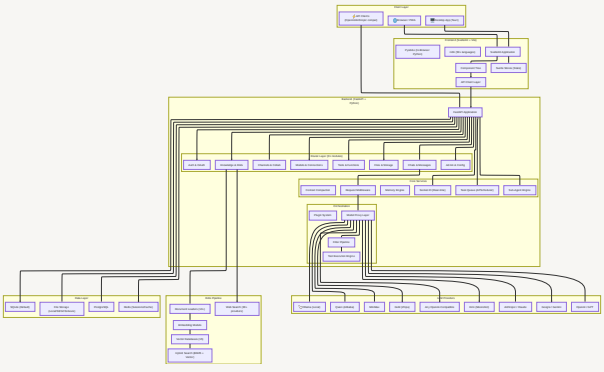
System Overview

Qwythos is a full-stack AI platform built on two primary pillars:

Layer	Technology	Role
Backend	Python 3.11 · FastAPI · SQLAlchemy · Alembic	API server, model orchestration, RAG, auth, real-time events
Frontend	SvelteKit · Vite · TypeScript · Tailwind CSS	Reactive UI, chat interface, admin dashboard, workspace tools

The platform connects to **any OpenAI-compatible API** and local model runners (Ollama), routing requests through a unified middleware layer that handles model selection, tool execution, context compaction, memory injection, and streaming response delivery.

High-Level Architecture Diagram



Philosophy & Design Principles

Qwythos embodies three core principles reflected in its Chinese philosophical naming:

Symbol	Meaning	System Mapping
理 (Reason)	Logical analysis, structured thinking	Claude, Kimi — deep reasoning and knowledge synthesis
想 (Imagine)	Creative generation, multimodal vision	GPT, Gemini, Qwen — versatile creative and visual capabilities
合 (Converge)	Unification, bringing together	GLM, MiniMax, The Stack — agentic automation and open weights

Design Principles:

1. **Model Agnosticism** — No vendor lock-in; any OpenAI-compatible API works
2. **Extensibility First** — Plugins, tools, functions, skills, filters, and actions
3. **Self-Hosted by Default** — Full offline operation capability
4. **Privacy Preserving** — Data stays on your infrastructure
5. **Progressive Enhancement** — Works as a simple chat, scales to enterprise orchestration

---

Backend Architecture

The backend is a Python FastAPI application located in `backend/qwythos/`.

Entry Point

- `main.py` — FastAPI app initialization, middleware stack, lifespan management, ASGI configuration (~2,879 lines)

Configuration System

File	Purpose
`config.py`	Runtime configuration with 500+ settings (~130K)
`env.py`	Environment variable resolution and defaults (~50K)
`constants.py`	Error messages, task constants

Router Layer (31 API Modules)

Every REST endpoint lives in `backend/qwythos/routers/`:

Router	File	Domain
Authentication	`auths.py`	Login, signup, OAuth, LDAP, SCIM, JWT
Chats	`chats.py`	Conversations, messages, history, fork, compact
Models	`models.py`	Model CRUD, connections, proxy configuration
Knowledge	`knowledge.py`	Knowledge bases, document upload, RAG config
Retrieval	`retrieval.py`	Search, embedding, vector operations (~133K)
Channels	`channels.py`	Real-time collaborative spaces
Files	`files.py`	File upload, storage, streaming
Tools	`tools.py`	Tool registration, execution, schemas
Functions	`functions.py`	User-defined Python functions
Skills	`skills.py`	Skill management and execution
Ollama	`ollama.py`	Ollama-specific model management
OpenAI	`openai.py`	OpenAI-compatible API proxy
Images	`images.py`	Image generation (DALL·E, ComfyUI, A1111)
Audio	`audio.py`	STT/TTS (Whisper, ElevenLabs, Azure)
Users	`users.py`	User profiles, preferences, activity
Groups	`groups.py`	Group management and permissions
Folders	`folders.py`	Chat organization and folder hierarchy
Notes	`notes.py`	Rich-text notes with AI chat
Memories	`memories.py`	Persistent cross-chat memory
Prompts	`prompts.py`	System prompt templates
Automations	`automations.py`	Scheduled task execution
Calendar	`calendar.py`	Calendar events and scheduling
Notifications	`notifications.py`	Webhook-based notification system
Evaluations	`evaluations.py`	Model arena, A/B testing, ELO
Analytics	`analytics.py`	Usage tracking, token consumption
Configs	`configs.py`	Admin configuration UI

Router	File	Domain
Tasks	`tasks.py`	Background task management
Terminals	`terminals.py`	Terminal server integration
Pipelines	`pipelines.py`	Pipeline plugin management
SCIM	`scim.py`	SCIM 2.0 user provisioning
Utils	`utils.py`	Utility endpoints

Data Models (26 ORM Models)

Located in `backend/qwythos/models/`, each mapping to a database table:

```
auths.py          → User authentication records
chats.py          → Chat conversations (100K+)
chat_messages.py  → Individual messages
channels.py       → Collaborative channels
models.py         → Model configurations
knowledge.py      → Knowledge base metadata
files.py          → File metadata and references
functions.py      → User-defined functions
tools.py          → Tool definitions
skills.py         → Skill configurations
prompts.py        → Prompt templates
notes.py          → Rich-text notes
memories.py       → Persistent memories
users.py          → User profiles
groups.py         → Group definitions
folders.py        → Folder hierarchy
automations.py    → Automation schedules
calendar.py       → Calendar events
feedbacks.py      → Model evaluation feedback
access_grants.py  → Sharing permissions
tags.py           → Chat tagging
config.py         → Persisted configuration
messages.py       → Channel messages
shared_chats.py   → Public chat sharing
prompt_history.py → Prompt usage history
oauth_sessions.py → OAuth session tracking
```

Utility & Middleware Layer (46 modules)

The `backend/qwythos/utils/` directory contains the core processing engines:

Module	Purpose	Size
`middleware.py`	**The brain** — Request/response processing, model orchestration, streaming, tool calls (~256K)	
`oauth.py`	Full OAuth 2.0/OIDC implementation (~102K)	
`tools.py`	Tool discovery, validation, execution (~67K)	
`anthropic.py`	Anthropic API compatibility layer	
`subagents.py`	Sub-agent spawning and lifecycle	
`memory.py`	Memory extraction and injection	
`context_compaction.py`	Long-conversation summarization	
`filter.py`	Inlet/outlet filter pipeline	
`plugin.py`	Plugin loading and execution	
`auth.py`	JWT, API key, session authentication	
`audit.py`	Request audit logging	
`mcp/client.py`	Model Context Protocol client	
`redis.py`	Redis session and cache management	
`notifications.py`	Multi-target notification dispatch	
`timers.py`	Chat timer scheduling	
`rate_limit.py`	Rate limiting engine	
`security_headers.py`	CSP, HSTS, frame options	

Frontend Architecture

The frontend is a SvelteKit application built with Vite, located in `src/`.

Application Shell

```
src/
├── app.html          → HTML template with meta, CSP, PWA manifest
├── app.css           → Global styles (~20K)
├── tailwind.css      → Tailwind configuration
├── app.d.ts          → TypeScript declarations
├── lib/              → Shared library code
│   ├── apis/         → Backend API clients (29 modules)
```

```
│   ├── components/   → UI components (12 directories, 9 root files)
│   ├── stores/       → Svelte reactive stores
│   ├── constants/    → Client-side constants
│   ├── types/        → TypeScript type definitions
│   ├── utils/        → Client utilities
│   ├── i18n/         → Internationalization (50+ languages)
│   ├── pyodide/      → In-browser Python execution
│   └── workers/      → Web Workers
└── routes/           → SvelteKit file-based routing
    ├── (app)/        → Main application routes
    │   ├── admin/    → Admin dashboard
    │   ├── c/        → Chat conversations
    │   ├── channels/ → Channel collaboration
    │   ├── calendar/ → Calendar views
    │   ├── notes/    → Notes workspace
    │   ├── workspace/→ Model/tool management
    │   ├── automations/ → Automation management
    │   ├── folders/  → Folder views
    │   ├── home/     → Home / landing
    │   └── playground/ → Model playground
    ├── auth/         → Login / signup
    ├── s/            → Shared chat pages
    └── preview/      → Preview pages
```

Component Hierarchy

API Client Layer

The `src/lib/apis/` directory mirrors the backend router structure with 29 TypeScript modules providing typed API access:

```
analytics/  audio/     auths/      automations/  calendar/
channels/   chats/     configs/    evaluations/  files/
folders/    functions/ groups/     images/       knowledge/
memories/   models/    notes/      notifications/ ollama/
openai/     prompts/   retrieval/  skills/       streaming/
terminal/   tools/     users/      utils/
```

Central API configuration lives in `index.ts` (~40K).

State Management

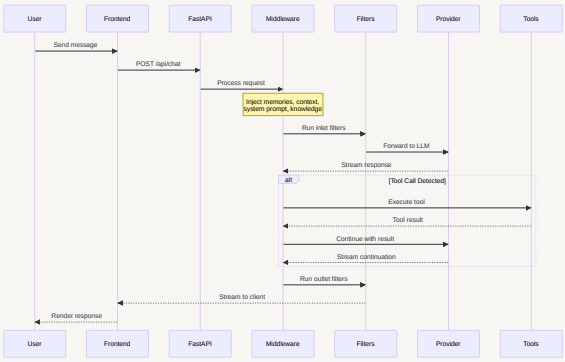
Svelte stores in `src/lib/stores/`:

- `index.ts` — Global application state (models, user, settings, theme, sidebar, etc.)
- `chatList.ts` — Chat list state with pagination and sorting

Multi-Model Orchestration

The orchestration layer is the heart of Qwythos — it routes requests to the right model(s), handles tool calling loops, and manages streaming responses.

Request Flow



Model Connection Types

Connection	Handler	Protocol
Ollama (Local)	`routers/ollama.py`	Ollama REST API
OpenAI-Compatible	`routers/openai.py`	OpenAI Chat Completions
Anthropic	`utils/anthropic.py`	Anthropic Messages API
Direct Passthrough	Connection settings	Raw request forwarding

Context Enhancement

Before each request reaches the LLM, the middleware enriches it with:

1. **System Prompt** — Model-specific instructions with variable interpolation

2. **Memory Injection** — Relevant stored memories (`utils/memory.py`)
3. **Knowledge Context** — RAG-retrieved documents
4. **Chat Variables** — User/chat-scoped values (`utils/chat_variables.py`)
5. **Context Compaction** — Summarized older messages for long conversations (`utils/context_compaction.py`)
6. **Tool Descriptions** — Available tool schemas

RAG & Retrieval Pipeline

The RAG system lives in `backend/qwythos/retrieval/` and provides full document-to-answer pipeline.

Document Ingestion

Content extractors in `retrieval/loaders/`:

Loader	Formats
`main.py`	PDF, DOCX, PPTX, XLSX, CSV, TXT, HTML, XML, JSON, YAML, MD, RST, ePub
`mistral.py`	Mistral OCR-powered extraction
`paddleocr_vl.py`	PaddleOCR visual-language extraction
`datalab_marker.py`	Datalab Marker document conversion
`mineru.py`	MinerU document processing
`microsoft_web_iq.py`	Microsoft Document Intelligence
`external_document.py`	External extraction engines
`external_web.py`	External web scraping
`youtube.py`	YouTube transcript extraction
`tavily.py`	Tavily web extraction

Vector Database Support (15 backends)

All implementations in `retrieval/vector/dbs/`:

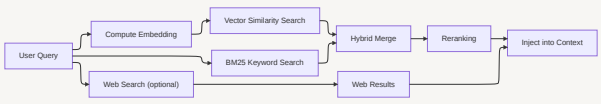
Database	File	Notes
ChromaDB	`chroma.py`	Default, embedded
PGVector	`pgvector.py`	PostgreSQL extension
Qdrant	`qdrant.py`	Standalone + multitenancy variant
Milvus	`milvus.py`	Standalone + multitenancy variant
Elasticsearch	`elasticsearch.py`	Full-text + vector
OpenSearch	`opensearch.py`	AWS-compatible
Pinecone	`pinecone.py`	Managed cloud
S3Vector	`s3vector.py`	S3-backed vector storage
Oracle 23ai	`oracle23ai.py`	Oracle database vectors
MariaDB Vector	`mariadb_vector.py`	MariaDB extension
openGauss	`opengauss.py`	Huawei database
Weaviate	`weaviate.py`	Graph-based vectors
Valkey	`valkey.py`	Redis-compatible vectors

Web Search Integration (30+ providers)

All implementations in `retrieval/web/`:

```
SearXNG     Google PSE    Brave Search    Kagi        DuckDuckGo
Tavily      Perplexity    Firecrawl       SerpStack   Serper
Serply      SearchApi     SerpApi         Bing        Jina
Exa         Sougou        Azure AI        Ollama      OpenSERP
Mojeek      LinkUp        Bocha           YaCy        Yandex
YDC         SerpHouse     BraveLLMContext Microsoft WebIQ
Perplexity Search         External
```

Search Pipeline



Real-Time Communication

Socket.IO powers all real-time features, implemented in `backend/qwythos/socket/`:

- `main.py` — Event handlers, room management, presence tracking (~40K)
- `utils.py` — Helper functions for broadcasting

Real-Time Features

- **Streaming responses** — Token-by-token delivery to the UI
- **Typing indicators** — Show when models are processing
- **Collaborative editing** — Shared document edits via CRDT (pycrdt)
- **Presence** — Online/offline/away user status
- **Channel messages** — Real-time collaborative chat
- **Notifications** — Push notifications to connected clients
- **Live updates** — Sidebar badges, unread counts, generation status

---

Tool & Plugin Execution Engine

Built-In Tools

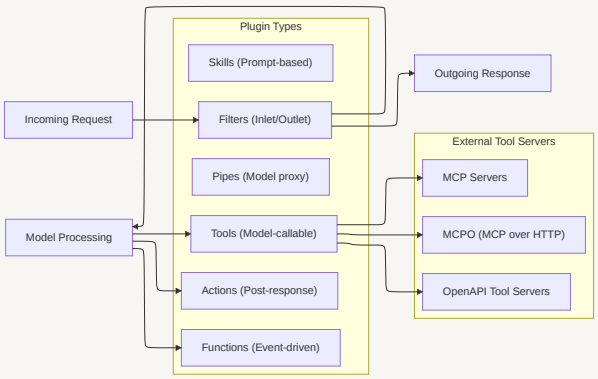
The built-in tool engine (`tools/builtin.py` — ~164K) provides:

- **Web Search** — Search the web and inject results
- **Code Interpreter** — Execute Python in-chat via Pyodide or terminal
- **Image Generation** — DALL·E, Gemini, ComfyUI, A1111
- **Knowledge Search** — Query attached knowledge bases
- **File Operations** — Read, search, and manage attached files
- **Calendar** — Create/query calendar events
- **Memory** — Store and recall user memories
- **Notifications** — Send notifications to the user
- **Timers** — Schedule delayed messages
- **Artifacts** — Persistent key-value storage

Knowledge File System

`tools/knowledge_fs.py` (~42K) provides file-system-like access to knowledge bases, enabling models to browse, search, and read documents programmatically.

Plugin Architecture



Security Model

Authentication Stack

Method	Module	Notes
Local accounts	`routers/auths.py`	Bcrypt + Argon2 password hashing
JWT tokens	`utils/auth.py`	PyJWT with cryptographic signing
OAuth 2.0 / OIDC	`utils/oauth.py`	Google, Microsoft, GitHub, custom OIDC
LDAP / Active Directory	`routers/auths.py`	With group synchronization
SCIM 2.0	`routers/scim.py`	Automated user provisioning
API Keys	`utils/auth.py`	Bearer token authentication
Trusted Headers	`utils/auth.py`	Reverse proxy authentication
PKCE	`utils/oauth.py`	Code challenge for all OAuth providers

Authorization

- **Role-Based Access Control (RBAC)** — Admin, User, Pending roles
- **Group Permissions** — Per-group model access, tool access, file quotas

- **Access Grants** — Sharing with read/write permissions
- **Model Access Control** — Per-model visibility (public/private/group)
- **Rate Limiting** — Per-user and per-endpoint rate limits (`utils/rate_limit.py`)

Security Headers

Managed by `utils/security_headers.py`:

- Content Security Policy (CSP) with configurable iframe sources
- HTTP Strict Transport Security (HSTS)
- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy

Storage & Data Layer

Database

Backend	Module	Use Case
SQLite	Built-in (aiosqlite)	Default, single-node, with optional encryption
PostgreSQL	psycopg3	Production, multi-node, full-text search

Schema migrations managed by **Alembic** in `backend/qwythos/migrations/`.

File Storage

Abstracted through `storage/provider.py`:

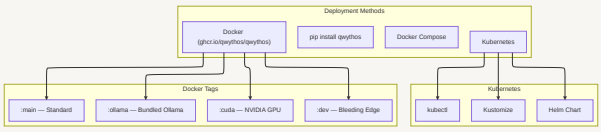
Provider	Notes
Local filesystem	Default
Amazon S3	Via boto3
Google Cloud Storage	Via google-cloud-storage
Azure Blob Storage	Via azure-storage-blob

Session & Cache

- **Redis** — Session management, WebSocket pub/sub, model list caching
- **Redis Sentinel** — HA Redis with automatic failover
- **In-memory** — Default for single-instance deployments

Infrastructure & Deployment

Deployment Options



Key Configuration Files

File	Purpose
`Dockerfile`	Multi-stage build (Node.js → Python)
`docker-compose.yaml`	Standard deployment
`docker-compose.gpu.yaml`	NVIDIA GPU passthrough
`docker-compose.amdgpu.yaml`	AMD GPU passthrough
`pyproject.toml`	Python package definition
`package.json`	Node.js dependencies & scripts
`svelte.config.js`	SvelteKit configuration
`vite.config.ts`	Vite build configuration
`railway.json`	Railway.app deployment

Observability

- **OpenTelemetry** — Traces, metrics, and logs via `utils/telemetry/`
- **Audit Logging** — Request-level audit trail via `utils/audit.py`
- **Usage Analytics** — Token consumption, model usage, user activity

---

Integration Points

Companion Projects

Project	Integration
Qwythos Computer	Mobile-first coding agent, connects as a model
Open Terminal / Terminals	Sandboxed code execution environment
oikb	Knowledge base sync from 45+ sources
Desktop App	Native Tauri app with Spotlight, push-to-talk

External Service Integration

Category	Services
LLM Providers	Ollama, OpenAI, Anthropic, Google, Mistral, Groq, OpenRouter, vLLM, LMStudio, and any OpenAI-compatible API
Speech	Whisper (local), OpenAI TTS, ElevenLabs, Azure Speech, Deepgram
Images	DALL·E, Gemini, ComfyUI, AUTOMATIC1111
Identity	Google, Microsoft, GitHub OAuth; LDAP/AD; SCIM (Okta, Azure AD)
Cloud Storage	Google Drive, OneDrive/SharePoint
Search	30+ web search providers (see RAG section)
Tool Servers	MCP, MCPO, OpenAPI-compatible servers

Directory Map

```
qwythos-bot/
├── backend/
│   └── qwythos/
│       ├── main.py              # FastAPI application entry point
│       ├── config.py            # 500+ runtime settings
│       ├── env.py               # Environment variable resolution
│       ├── events.py            # Application event hooks
│       ├── constants.py         # Error messages & constants
│       ├── functions.py         # Function execution sandbox
│       ├── tasks.py             # Background task scheduler
│       ├── routers/             # 31 API route modules
│       ├── models/              # 26 SQLAlchemy ORM models
│       ├── utils/               # 46 utility/middleware modules
```

```
│       ├── retrieval/           # RAG pipeline
│       │   ├── loaders/         # 10 document extractors
│       │   ├── vector/dbs/      # 15 vector DB backends
│       │   └── web/             # 30+ web search providers
│       ├── tools/               # Built-in tool engine
│       ├── socket/              # Socket.IO real-time
│       ├── storage/             # File storage abstraction
│       ├── internal/            # Internal services
│       └── migrations/          # Alembic DB migrations
├── src/
│   ├── app.html                 # HTML shell
│   ├── app.css                  # Global styles
│   └── lib/
│       ├── apis/                # 29 API client modules
│       ├── components/          # UI component tree
│       │   ├── chat/            # Chat interface (120K+ main)
│       │   ├── channel/         # Channels
│       │   ├── admin/           # Admin panel
│       │   ├── workspace/       # Workspace management
│       │   ├── notes/           # Notes editor
│       │   ├── calendar/        # Calendar
│       │   ├── common/          # Shared components
│       │   ├── layout/          # App layout
│       │   └── icons/           # Icon components
│       ├── stores/              # Svelte state stores
│       ├── i18n/                # 50+ language translations
│       ├── types/               # TypeScript definitions
│       ├── utils/               # Client utilities
│       └── pyodide/             # In-browser Python
├── docs/                        # Documentation
├── desktop/                     # Desktop app (Tauri)
├── build/                       # Build output
├── static/                      # Static assets
├── bench/                       # Benchmarks
├── test/                        # Test suite
└── scripts/                     # Build & utility scripts
```

> **Qwythos** — *Where seven minds converge into infinite possibility.*